

# **Programming Language Constructs A Locally Optimizing Compiler**

Peter Sestoft  
Niels Hallenberg  
2026-07-24

# Today

- Deficiencies in the old micro-C compiler
- A backwards, continuation-based, compiler
- Tail calls
- Optimizing code on the fly (peephole optimizations)
- File used: `Contcomp.fs` found in folder `MicroC`.

# Micro-C compiler shortcomings

- The compiler often generates **inefficient code**

GETBP

CSTI 0

ADD

LDI

could  
be

GETBP

LDI

INCSP -1

INCSP -1

could  
be

INCSP -2

- The **compiler itself is inefficient**, using (@) a lot:

```
| If(e, stmt1, stmt2) ->
  let labelse = newLabel()
  let labend = newLabel()
  in cExpr e varEnv funEnv @ [IFZERO labelse]
    @ cStmt stmt1 varEnv funEnv @ [GOTO labend]
    @ [Label labelse] @ cStmt stmt2 varEnv funEnv
    @ [Label labend]
```

- Tail calls** are not executed in constant space

# Example, if-statement (ex19.c)

```
void main(int x) {  
    if (x == 0) print 33; else print 44;  
}
```

- The old micro-C compiler produces this:

```
    GETBP; CSTI 0; ADD; LDI; CSTI 0; EQ; IFZERO L2;  
    CSTI 33; PRINTI; INCSP -1; GOTO L3;  
L2: CSTI 44; PRINTI; INCSP -1;  
L3: INCSP 0; RET 0
```

- We want it to produce this better code:

```
    GETBP; LDI; IFNZRO L2;  
    CSTI 33; PRINTI; RET 1;  
L2: CSTI 44; PRINTI; RET 1
```

# Generating code backwards

```
cExpr expr varEnv funEnv : instr list
```

OLD

```
cExpr expr varEnv funEnv C : instr list
```

NEW

- C is the code following the code for expr
- That is, C represents the continuation of expr
- Code is generated backwards
- Why is this useful?
  - Code for expr “knows what happens next”
  - So can optimize code for expr; if C is [NOT] and expr is 1, then [CSTI 1; NOT] should be optimized to [CSTI 0]
  - The compiler avoids the expensive @ operations

# The old and new expression compiler

Comp.fs

```
and cExpr e varEnv funEnv : instr list =  
  match e with  
  | CstI i          -> [CSTI i]  
  | Prim1(ope, e1) ->  
    cExpr e1 varEnv funEnv  
    @ (match ope with  
        | "!"          -> [NOT]  
        | "printi"     -> [PRINTI] ...)
```

OLD

Make lists of  
instructions,  
append them

```
and cExpr e varEnv funEnv C : instr list =  
  match e with  
  | ...  
  | CstI i          -> CSTI i :: C  
  | Prim1(ope, e1) ->  
    cExpr e1 varEnv funEnv  
    (match ope with  
      | "!"          -> NOT :: C  
      | "printi"     -> PRINTI :: C  
      | ...)
```

NEW

Put new code  
in front of  
given code

Contcomp.fs

NB: Same code  
so far, generated  
more efficiently



# Compiling Blocks in 2 passes (part 1)

```
void main(int n) {  
    int i;  
    i=0;  
    print i;  
    int j;  
    j=42;  
    print j;  
    println;  
}
```

- We want to compile the block backwards.
- We have a mixture of declarations and statements.
- The statement `print i` must be compiled in an environment with `i` declared, but not `j`, etc.
- Doing this forwardly, as in `Comp.fs`, is straight forward.
- In `Contcomp.fs`, we need two passes; one to compute environments forwardly and one to compute statements backwards.



# Compiling Blocks in 2 passes (part 2)

```
let rec cStmt stmt (varEnv : varEnv) (funEnv : funEnv)
    (C : instr list) : instr list =
...
| Block stmts ->
    let rec pass1 stmts ((_, fdepth) as varEnv) =
        match stmts with
and bStmtordec stmtOrDec varEnv : bStmtordec * varEnv =
    match stmtOrDec with
    | Stmt stmt ->
        (BStmt stmt, varEnv)
    | Dec (typ, x) ->
        let (varEnv1, code) = allocate Locvar (typ, x) varEnv
        (BDec code, varEnv1)
    | [] -> C
    | (BDec code, varEnv)::sr->code @ pass2 sr C
    | (BStmt stmt, varEnv)::sr->cStmt stmt varEnv funEnv (pass2 sr C)
pass2 stmtsback (addINCSP(snd varEnv - fdepthend) C)
| ...

type bStmtordec =
    | BDec of instr list
    | BStmt of stmt
```

A forward pass over declarations and statements. Generate code for declarations and build environments.

A backwards pass to compile statements with environments from pass 1.



# Code improvement rules based on bytecode equivalences

| Code             | Better equivalent code | When       |
|------------------|------------------------|------------|
| 0; ADD           | <no code>              |            |
| 0; SUB           | <no code>              |            |
| 0; NOT           | 1                      |            |
| n; NOT           | 0                      | $n \neq 0$ |
| 1; MUL           | <no code>              |            |
| 1; DIV           | <no code>              |            |
| 0; EQ            | NOT                    |            |
| INCSP 0          | <no code>              |            |
| INCSP m; INCSP n | INCSP (m+n)            |            |
| 0; IFZERO a      | GOTO a                 |            |
| n; IFZERO a      | <no code>              | $n \neq 0$ |
| 0; IFNZRO a      | <no code>              |            |
| n; IFNZRO a      | GOTO a                 | $n \neq 0$ |

# Naive code from Comp.fs (ex13.c)

```
void main(int n) {  
    int y;  
    y = 1889;  
    while (y < n) {  
        y = y + 1;  
        if (y % 4 == 0 && (y % 100 != 0 || y % 400 == 0))  
            print y;  
    }  
}
```

source

generated by  
Comp.fs compiler

```
INCSP 1; GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;  
L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI; CSTI 1; ADD; STI;  
INCSP -1; GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; CSTI 0; EQ;  
IFZERO L7; GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; CSTI 0; EQ; NOT;  
IFNZRO L9; GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; CSTI 0; EQ;  
GOTO L8; L9: CSTI 1; L8: GOTO L6; L7: CSTI 0; L6: IFZERO L4; GETBP;  
CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L5; L4: INCSP 0; L5:  
INCSP 0; L3: GETBP; CSTI 1; ADD; LDI; GETBP; CSTI 0; ADD; LDI; LT;  
IFNZRO L2; INCSP -1; RET 0
```

- Let's simplify the bytecode by hand, e.g., by deleting superfluous instructions

# Joint exercise 1 (code for ex13.c)

```
INCSP 1;
GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;

L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;

GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; CSTI 0; EQ; IFZERO L7;

GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; CSTI 0; EQ; NOT; IFNZRO L9;

GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; CSTI 0; EQ; GOTO L8;

L9: CSTI 1;
L8: GOTO L6;
L7: CSTI 0;
L6: IFZERO L4;
GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L5;

L4: INCSP 0;
L5: INCSP 0;
L3: GETBP; CSTI 1; ADD; LDI; GETBP; CSTI 0; ADD; LDI; LT; IFNZRO L2;

INCSP -1; RET 0
```

Allocate y. n has index 0 and y has index 1

y = 1889

y = y + 1

if y%4==0 goto L7

if y % 100 != 0 goto L9

y % 400 == 0 goto L8

add 1 (true) on the stack

add 0 (false) on the stack

if zero, goto else block which is empty

print y and remove the value that printi leaves on the stack.

Empty else block

Done with while block having no locals

while y < n goto L2

deallocate y and return from function

# Joint exercise 1 (code for ex13.c)

```
INCSP 1;
GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;

L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;

GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; NOT; IFZERO L7;

GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; NOT; NOT; IFNZRO L9;

GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; NOT; GOTO L8;

L9: CSTI 1;
L8: GOTO L6;
L7: CSTI 0;
L6: IFZERO L4;
GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L5;

L4: INCSP 0;
L5: INCSP 0;
L3: GETBP; CSTI 1; ADD; LDI; GETBP;
    LDI; LT; IFNZRO L2;

INCSP -1; RET 0
```

Allocate y. n has index 0 and y has index 1

y = 1889

y = y + 1

if y%4==1 goto L7

if y % 100 != 0 goto L9

if zero, goto else block which is empty

print y and remove the value that printi leaves on the stack.

while y < n goto L2

deallocates y and return from function

add 1 (true) on the stack

add 0 (false) on the stack

Empty else block

Done with while block having no locals

# Optimizing code while generating it

```
and cExpr e varEnv funEnv C : instr list =  
  match e with  
  | CstI i          -> addCST i C
```

```
let rec addCST i C =  
  match (i, C) with  
  | (0, ADD      :: C1) -> C1  
  | (0, SUB      :: C1) -> C1  
  | (0, NOT      :: C1) -> addCST 1 C1  
  | (_, NOT      :: C1) -> addCST 0 C1  
  | (1, MUL      :: C1) -> C1  
  | (1, DIV      :: C1) -> C1  
  | (0, EQ       :: C1) -> addNOT C1  
  | (0, IFZERO lab :: C1) -> addGOTO lab C1  
  | (_, IFZERO lab :: C1) -> C1  
  | ...  
  | _             -> CSTI i :: C
```

If no optimization possible, generate CSTI instruction

# Some examples

GETBP; 0; ADD; LDI



GETBP; LDI

<x>; 0; EQ; IFZERO a



<x>; NOT; IFZERO a

| <x> | 0 | EQ |
|-----|---|----|
| 0   | 0 | 1  |
| <>0 | 0 | 0  |

| <x> | NOT |
|-----|-----|
| 0   | 1   |
| <>0 | 0   |

# Optimizing negations before tests

| Code          | Better equivalent code |
|---------------|------------------------|
| NOT; NOT      | <no code>              |
| NOT; IFZERO a | IFNZRO a               |
| NOT; IFNZRO a | IFZERO a               |

```
let addNOT C =  
  match C with  
  | NOT          :: C1 -> C1  
  | IFZERO lab   :: C1 -> IFNZRO lab :: C1  
  | IFNZRO lab   :: C1 -> IFZERO lab  :: C1  
  | _            -> NOT  :: C
```

<x>; NOT; IFZERO a



<x>; IFNZRO a

# Joint exercise 2 (code for ex13.c)

```
INCSP 1;
GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;

L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;

GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; NOT; IFZERO L7;

GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; NOT; NOT; IFNZRO L9;

GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; NOT; GOTO L8;

L9: CSTI 1;
L8: GOTO L6;
L7: CSTI 0;
L6: IFZERO L4;
GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L5;

L4: INCSP 0;
L5: INCSP 0;
L3: GETBP; CSTI 1; ADD; LDI; GETBP; LDI; LT; IFNZRO L2;

INCSP -1; RET 0
```

Allocate y. n has index 0 and y has index 1

y = 1889

y = y + 1

if y%4==1 goto L7

if y % 100 != 0 goto L9

if zero, goto else block which is empty

print y and remove the value that printi leaves on the stack.

while y < n goto L2

deallocate y and return from function

add 1 (true) on the stack

add 0 (false) on the stack

Empty else block

Done with while block having no locals

# Joint exercise 2 (code for ex13.c)

```
INCSP 1;
GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;

L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;

GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; IFNZRO L7;

GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; IFNZRO L9;

GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; NOT; GOTO L8;

L9: CSTI 1;
L8: GOTO L6;
L7: CSTI 0;
L6: IFZERO L4;

GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L5;

L4: INCSP 0;
L5: INCSP 0;
L3: GETBP; CSTI 1; ADD; LDI; GETBP; LDI; LT; IFNZRO L2;

INCSP -1; RET 0
```

Allocate y. n has index 0 and y has index 1

y = 1889

y = y + 1

if y%4==1 goto L7

IFNZRO L7;

if y % 100 != 0 goto L9

if y % 400 == 0 and goto L8

add 1 (true) on the stack

add 0 (false) on the stack

if zero, goto else block which is empty

print y and remove the value that printi leaves on the stack.

Empty else block

Done with while block having no locals

while y < n goto L2

deallocate y and return from function

# Optimizing stack pointer updates

| Code               | Better equivalent code |
|--------------------|------------------------|
| INCSP m1; INCSP m2 | INCSP (m1+m2)          |
| INCSP m1; RET m2   | RET (m2-m1)            |
| INCSP 0            | <no code>              |

```
let rec addINCSP m1 C : instr list =  
  match C with  
  | INCSP m2          :: C1 -> addINCSP (m1+m2) C1  
  | RET m2            :: C1 -> RET (m2-m1) :: C1  
  | Label lab :: RET m2 :: _ -> RET (m2-m1) :: C  
  | _ -> if m1=0 then C else INCSP m1 :: C
```

Becomes RET(m2-m1)::Label lab::RET m2::...

INCSP 0; RET 0



RET 0

INCSP -1; RET 0



RET 1

# Joint exercise 3 (code for ex13.c)

```
INCSP 1;
GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;

L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;

GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; IFNZRO L7;

GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; IFNZRO L9;

GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; NOT; GOTO L8;

L9: CSTI 1;
L8: GOTO L6;
L7: CSTI 0;
L6: IFZERO L4;
GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L5;

L4: INCSP 0;
L5: INCSP 0;
L3: GETBP; CSTI 1; ADD; LDI; GETBP; LDI; LT; IFNZRO L2;

INCSP -1; RET 0;
```

Allocate y. n has index 0 and y has index 1

y = 1889

y = y + 1

if y%4==1 goto L7

if y % 100 != 0 goto L9

add 1 (true) on the stack

y % 400 == 0 and goto L8

add 0 (false) on the stack

if zero, goto else block which is empty

print y and remove the value that printi leaves on the stack.

Empty else block

Done with while block having no locals

while y < n goto L2

deallocate y and return from function

# Joint exercise 3 (code for ex13.c)

```
INCSP 1;
GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;

L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;

GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; IFNZRO L7;

GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; IFNZRO L9;

GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; NOT; GOTO L8;

L9: CSTI 1;
L8: GOTO L6;
L7: CSTI 0;
L6: IFZERO L4;
GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L5;

L4:
L5:
L3: GETBP; CSTI 1; ADD; LDI; GETBP; LDI; LT; IFNZRO L2;

RET 1
```

Allocate y. n has index 0 and y has index 1

y = 1889

y = y + 1

if y%4==1 goto L7

if y % 100 != 0 goto L9

add 1 (true) on the stack

y % 400 == 0 and goto L8

add 0 (false) on the stack

if zero, goto else block which is empty

print y and remove the value that printi leaves on the stack.

Empty else block

Done with while block having no locals

while y < n goto L2

deallocate y and return from function

# Avoiding jumps to jumps

- A conditional jump (IFZERO, IFNZRO) to some code, `<stmt2>`, needs a label on that code, but
  - if the code has a label already, use that
  - if the code starts with a GOTO lab, use lab

```
let addLabel C : label * instr list =  
  match C with  
  | Label lab :: _ -> (lab, C)  
  | GOTO lab :: _ -> (lab, C)  
  | _ -> let lab = newLabel()  
         in (lab, Label lab :: C)
```

```
<e> IFZERO L1  
<stmt1> GOTO L2  
L1: <stmt2>  
L2: . . .
```

The label  
returned by  
addLabel

The continuation C when  
addLabel is called.

```
<e> IFZERO L2  
<stmt1> GOTO L2  
L1: GOTO L2  
L2: . . .
```

The label  
returned by  
addLabel

The continuation C when  
addLabel is called.

# Avoiding jumps to jumps and returns

- An unconditional jump (GOTO) to some code needs a label on the code, but
  - If the code has a label already, use that
  - If the code starts with a GOTO lab, use lab
  - If the code executes RET m, just do RET m

```
let makeJump C : instr * instr list =  
  match C with  
  | RET m                :: _ -> (RET m, C)  
  | Label lab :: RET m   :: _ -> (RET m, C)  
  | Label lab           :: _ -> (GOTO lab, C)  
  | GOTO lab            :: _ -> (GOTO lab, C)  
  | _ -> let lab = newLabel()  
        in (GOTO lab, Label lab :: C)
```

```
<e> IFZERO L1  
<stmt1> GOTO L2  
L1: <stmt2>  
L2: . .
```

The jump  
instruction  
returned by  
makeJump

The continuation C after  
the if-statement when  
makeJump is called.

```
<e> IFZERO L1  
<stmt1> RET 3  
L1: <stmt2>  
L2: RET 3
```

The jump  
instruction  
returned by  
makeJump

The continuation C after  
the if-statement when  
makeJump is called.

# Eliminating dead code

- Dead code is code that cannot be executed:

```
GOTO L1; CSTI 42; PRINTI; INCSP -1; L1:
```

Dead code

- Code following GOTO or RET is dead unless preceded by a label:

```
let rec deadcode C =  
  match C with  
  | [] -> []  
  | Label lab :: _ -> C  
  | _ :: C1 -> deadcode C1
```

Discard  
dead code

# Adding a jump instruction (addJump)

- Naïve compilation of if statement without else branch

```
if (e)  
  stmt
```

```
<e> IFZERO L2  
<stmt> GOTO L2  
L2:
```

Really stupid code – jumping to the next instruction

```
let addJump jump C =  
  let C1 = deadcode C  
  match (jump, C1) with  
  | (GOTO lab1, Label lab2 :: _) ->  
    if lab1=lab2 then C1  
    else GOTO lab1 :: C1  
  | _ -> jump :: C1
```

We are adding an unconditional jump and can therefore remove all code till next label.

# Compilation of if-statements

```
if (e)
  s1
else
  s2
```

```
<e>
IFZERO L1
<s1>
GOTO L2
L1: <s2>
L2:
```

```
let rec cStmt stmt varEnv funEnv C : instr list =
  match stmt with
  | If(e, stmt1, stmt2) ->
    let (jumpend, C1) = makeJump C to L2
    let (labelse, C2) =
      addLabel (cStmt stmt2 varEnv funEnv C1) L1
    in cExpr e varEnv funEnv
      (IFZERO labelse
       :: cStmt stmt1 varEnv funEnv
         (addJump jumpend C2))
```

# Compilation of while-statements

```
while (e)
  s
```

```
GOTO L2
L1: <s>
L2: <e>
    IFNZRO L1
```

```
let rec cStmt stmt varEnv funEnv C : instr list =
  match stmt with
  | While(e, body) ->
    let labbegin = newLabel()
    let (jumptest, C1) =
      makeJump (cExpr e varEnv funEnv
                    (IFNZRO labbegin :: C))
    in addJump jumptest (Label labbegin
                        :: cStmt body varEnv funEnv C1)
```



# Joint exercise 4 (code for ex13.c)

```
INCSP 1;
GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;

L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;

GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; IFNZRO L7;

GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; IFNZRO L9;

GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; NOT; GOTO L8;

L9: CSTI 1;
L8: GOTO L6;
L7: CSTI 0;
L6: IFZERO L4;

GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L5;

L4:
L5:
L3: GETBP; CSTI 1; ADD; LDI; GETBP; LDI; LT; IFNZRO L2;

RET 1
```

Allocate y. n has index 0 and y has index 1

y = 1889

y = y + 1

Jump directly to GETBP after L6

Jump directly to L6

If y % 100 != 0 goto L9

y % 400 == 0 and goto L8

add 1 (true) on the stack

add 0 (false) on the stack

if zero, goto else block which is empty

print y and remove the value that printi leaves on the stack.

Empty else block

Done with while block having no locals

while y < n goto L2

deallocate y and return from function

# Joint exercise 4 (code for ex13.c)

```
INCSP 1;
GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;

L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;

GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; IFNZRO L7;

GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; IFNZRO L10;

GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; NOT; GOTO L6;

L7: CSTI 0;
L6: IFZERO L4;
L10: GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L3;

L4:
L5:
L3: GETBP; CSTI 1; ADD; LDI; GETBP; LDI; LT; IFNZRO L2;

RET 1
```

Allocate y. n has index 0 and y has index 1

y = 1889

y = y + 1

Jump directly to GETBP after L6

Jump directly to L6

If y % 100 != 0 goto L9

y % 400 == 0 and goto L8

add 1 (true) on the stack

add 0 (false) on the stack

if zero, goto else block which is empty

print y and remove the value that printi leaves on the stack.

Empty else block

Done with while block having no locals

while y < n goto L2

deallocate y and return from function

# Joint exercise 5 (code for ex13.c)

```
INCSP 1;
GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;

L2: GETBP; CSTI 1; ADD; GETBP; LDI;
    CSTI 1; ADD; STI; INCSP -1;

GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; IFNZRO L7;

GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; IFNZRO L10;

GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; NOT; GOTO L6;

L7: CSTI 0;
L6: IFZERO L4;
L10: GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1; GOTO L5;

L4:
L5:
L3: GETBP; CSTI 1; ADD; LDI; GETBP; LDI; LT; IFNZRO L2;

RET 1
```

Allocate y. n has index 0 and y has index 1

y = 1889

Jump directly to L4

y = y + 1

if y%4==1 goto L7

Replace with IFZERO L4

if y % 100 != 0 goto L9

add 1 (true) on the stack

y % 400 == 0 and goto L8

add 0 (false) on the stack

if zero, goto else block which is empty

Empty else block

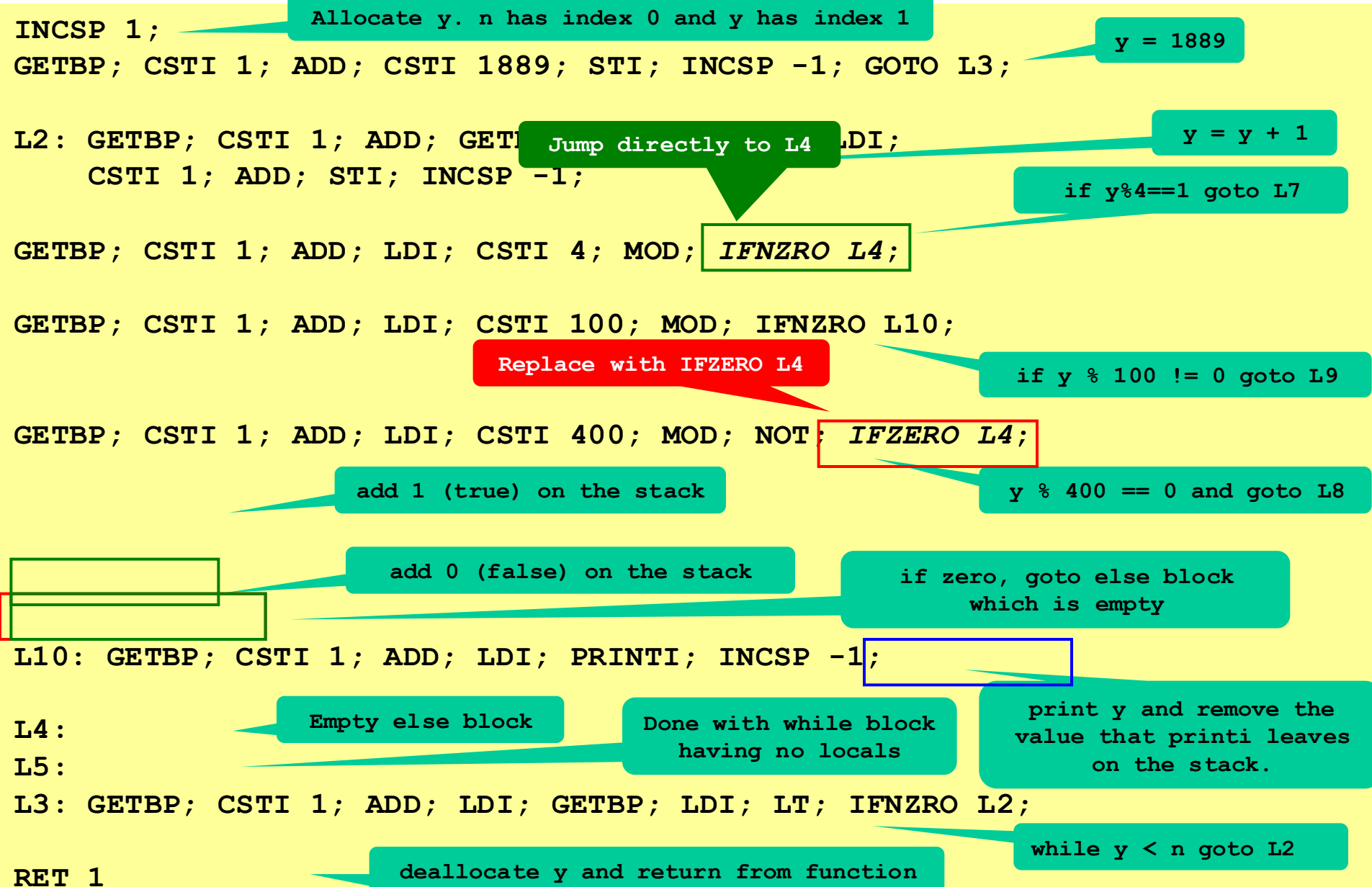
Done with while block having no locals

print y and remove the value that printi leaves on the stack.

while y < n goto L2

deallocate y and return from function

# Joint exercise 5 (code for ex13.c)



# Joint exercise 5 (code for ex13.c)

Optimized by hand:

```
INCSP 1; GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO L3;
L2: GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;
GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; IFNZRO L4;
GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; IFNZRO L10;
GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; NOT; IFZERO L4;
L10: GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1;
L4:
L5:
L3: GETBP; CSTI 1; ADD; LDI; GETBP; LDI; LT; IFNZRO L2;
RET 1
```

Generated by Contcomp.fs:

```
INCSP 1; GETBP; CSTI 1; ADD; CSTI 1889; STI; INCSP -1; GOTO "L3";
Label "L2"; GETBP; CSTI 1; ADD; GETBP; CSTI 1; ADD; LDI;
    CSTI 1; ADD; STI; INCSP -1;
GETBP; CSTI 1; ADD; LDI; CSTI 4; MOD; IFNZRO "L3";
GETBP; CSTI 1; ADD; LDI; CSTI 100; MOD; IFNZRO "L4";
GETBP; CSTI 1; ADD; LDI; CSTI 400; MOD; IFNZRO "L3";
Label "L4"; GETBP; CSTI 1; ADD; LDI; PRINTI; INCSP -1;
Label "L3"; GETBP; CSTI 1; ADD; LDI; GETBP; LDI; LT; IFNZRO "L2";
RET 1
```



# Compiling shortcut logical expressions

- Logical expression ( $m==0 \ \&\& \ n==0$ ) may
  - produce a value, as in `b = (m==0&& n==0) ;`
  - decide a test, as in `if (m==0&&n==0) ...`

```
<e1>  
IFZERO L1  
<e2>  
GOTO L2
```

```
L1: 0
```

```
L2:
```

Standard code for  
value of  $e1 \&\& e2$

```
<e1>  
IFZERO L1  
<e2>  
GOTO L2
```

```
L1: 0
```

```
L2: IFZERO L3
```

When used  
in **if** (...) ...

... we can  
optimize it

```
<e1>  
IFZERO L3  
<e2>  
IFZERO L3
```



# Compiling shortcut logical expressions

- Logical expression ( $m==0 \ \&\& \ n==0$ ) may
  - decide a test, as in `while (m==0&& n==0) ...`

```
      GOTO L2
L1:   <s>
L2:   <e1>
      IFZERO L3
      <e2>
      GOTO L4
L3:   0
L4:   IFNZERO L1
```

When used in  
**while** (...) ...



```
      GOTO L2
L1:   <s>
L2:   <e1>
      IFZERO L7
      <e2>
      IFNZERO L1
L7:
```

... we can  
optimize it

# Compiling e1 && e2

```
and cExpr e varEnv funEnv C : instr list =  
  match e with  
  | Andalso(e1, e2) ->  
    match C with  
    | IFZERO lab :: _ ->  
      cExpr e1 varEnv funEnv  
        (IFZERO lab :: cExpr e2 varEnv funEnv C)  
      if (e1&&e2)  
    | IFNZRO labthen :: C1 ->  
      let (labelse, C2) = addLabel C1  
      in cExpr e1 varEnv funEnv  
        (IFZERO labelse  
         :: cExpr e2 varEnv funEnv (IFNZRO labthen :: C2))  
      while (e1&&e2)  
    | _ ->  
      let (jumpend, C1) = makeJump C  
      let (labfalse, C2) = addLabel (addCST 0 C1)  
      in cExpr e1 varEnv funEnv  
        (IFZERO labfalse  
         :: cExpr e2 varEnv funEnv (addJump jumpend C2))  
      b = (e1&&e2)  
  | Orelse(e1, e2) -> ... dual to Andalso ...
```

# Compiling tail calls

- Why *tail calls*?
- A call is a *tail call* if it is the last action of a function, e.g. ex12.c:

```
int main(int n) {  
    if (n)  
        return main(n-1);  
    else  
        return 17;  
}
```

Tail call; nothing  
to do after it

- In the code, a tail call is followed by RET:

```
L1: GETBP; CSTI 0; ADD; LDI; IFZERO L2;  
    GETBP; CSTI 0; ADD; LDI; CSTI 1; SUB;  
    CALL L1; RET 1; GOTO L3  
L2: CSTI 17; RET 1;  
L3: INCSP 0; RET 0
```

Tail call

# Stack machine execution of TCALL

```
int main() { ... g(...) ... }
int g(...) { return f(...); }
int f(...) { return ...; }
```

main calls g

g calls f

f returns

## Ordinary call and two returns

|  |             |            |                 |
|--|-------------|------------|-----------------|
|  | ret<br>addr | old<br>bp1 | g's vars<br>etc |
|--|-------------|------------|-----------------|

CALL f

|  |             |            |                 |             |            |                 |
|--|-------------|------------|-----------------|-------------|------------|-----------------|
|  | ret<br>addr | old<br>bp1 | g's vars<br>etc | ret<br>addr | old<br>bp2 | f's vars<br>etc |
|--|-------------|------------|-----------------|-------------|------------|-----------------|

RET (from f)

|  |             |            |                 |
|--|-------------|------------|-----------------|
|  | ret<br>addr | old<br>bp1 | g's vars<br>etc |
|--|-------------|------------|-----------------|

RET (from g)

|  |
|--|
|  |
|--|

## Tail call and one return

|  |             |            |                 |
|--|-------------|------------|-----------------|
|  | ret<br>addr | old<br>bp1 | g's vars<br>etc |
|--|-------------|------------|-----------------|

TCALL f

|  |             |            |                 |
|--|-------------|------------|-----------------|
|  | ret<br>addr | old<br>bp1 | f's vars<br>etc |
|--|-------------|------------|-----------------|

RET (from f)

|  |
|--|
|  |
|--|

- Return from tail call goes directly to original caller



# Micro-C machine call and return

- CALL creates a stack frame
- RET destroys a stack frame
- TCALL destroys one frame and creates another:

80 CALL  $m\ a$   $s, v_1, \dots, v_m \Rightarrow s, \boxed{r, bp, v_1, \dots, v_m}$

81 TCALL  $m\ n\ a$   $s, \boxed{r, b, u_1, \dots, u_n}, v_1, \dots, v_m \Rightarrow s, \boxed{r, b, v_1, \dots, v_m}$

86 RET  $m$   $s, \boxed{r, b, v_1, \dots, v_m}, v \Rightarrow s, v$

- There is nothing to do after tail call, except return
- So the caller's stack frame can be discarded before the tail call
- Tail calls do not require unbounded stack space

# Recognizing tail calls in the compiler

- To call a function, compile arguments and emit a call:

```
and callfun f es varEnv funEnv C : instr list =  
    ...  
    cExprs es varEnv funEnv (makeCall argc labf C)
```

- A tail call is a call followed by RET
- Easy to see when generating code backwards:

```
let makeCall m lab C : instr list =  
    match C with  
    | RET n                :: C1 -> TCALL(m, n, lab) :: C1  
    | Label1 _ :: RET n :: _ -> TCALL(m, n, lab) :: C  
    | _                    -> CALL(m, lab) :: C
```

# The effect of optimizations

New code for ex12.c

```
L1: GETBP; LDI; IFZERO L2;  
    GETBP; LDI; CISTI 1; SUB; TCALL (1,1,"L1");  
L2: CISTI 17; RET 1
```

- Compiling ex11.c with old and new comp.
- Finding 14200 solutions to 12-queen puzzle

|                           | <b>Code size<br/>(instr)</b> | <b>Running<br/>time (sec.)</b> |
|---------------------------|------------------------------|--------------------------------|
| Old direct compiler       | 792                          | 9.06                           |
| New backwards<br>compiler | 701                          | 8.00                           |

# Tail call optimization

- Java does not optimize tail calls
  - And JVM does not optimize tail calls
  - The security model requires stack inspection
- C# does not optimize tail calls
  - But .NET/CLI bytecode supports tail calls

```
IL_000e:  tail.  
IL_0010:  callvirt int32 MyClass::MyMethod(int32)
```

- Scheme, ML, F# ... optimize tail calls
  - Needed because loops are written using tail calls

```
let rec sum xs acc =  
  match xs with  
  | []      -> acc  
  | x::xr   -> sum xr (x+acc) ;;
```